



UNIVERZITET U NIŠU
ELEKTRONSKI FAKULTET



REPLIKACIJA I MEHANIZMI VISOKE DOSTUPNOSTI U MONGODB REPLICATION SET ARHITEKTURI

- SEMINARSKI RAD -

Studijski program: Računarstvo
i informatika

Modul: Bezbednost računarskih
sistema

Predmet: Sistemi za upravljanje
bazama podataka

Student:

Andrea Valčić, br. Ind. 1928

Profesor:

prof. dr Aleksandar Stanimirović

Niš, Jul 2026. godine

SADRŽAJ

1. UVOD	3
2. MONGODB KAO NOSQL SISTEM	4
2.1. NOSQL BAZE PODATAKA	4
2.2. KARAKTERISTIKE MONGODB SISTEMA	4
2.3. DOKUMENTNI MODEL I BSON FORMAT	5
2.4. PREDNOSTI I OGRANIČENJA MONGODB SISTEMA	6
3. REPLICA SET ARHITEKTURA	6
3.1. OSNOVE REPLICA SET KONCEPTA	6
3.2. ULOGE ČVOROVA U REPLICA SET SISTEMU	7
3.3. MEHANIZAM REPLIKACIJE I OPLOG	8
3.4. HEARTBEAT MEHANIZAM I PRAĆENJE ZDRAVLJA ČVOROVA	9
3.5. PROCES IZBORA PRIMARNOG ČVORA (ELECTION)	9
3.6. KONFIGURACIJA I PRIORITETI ČVOROVA	10
4. MEHANIZMI VISOKE DOSTUPNOSTI.....	11
4.1. POJAM VISOKE DOSTUPNOSTI	11
4.2. AUTOMATSKI FAILOVER.....	11
4.3. READ PREFERENCE	12
4.4. WRITE CONCERN	13
4.5. READ CONCERN	14
4.6. OPORAVAK SISTEMA NAKON FAILOVERA	14
5. PREDNOSTI I OGRANIČENJA REPLICA SET ARHITEKTURE	15
5.1. PREDNOSTI.....	15
5.2. OGRANIČENJA	16
5.3. KADA KORISTITI REPLICA SET ARHITEKTURU	17
6. IMPLEMENTACIJA PRAKTIČNOG DELA	17
6.1. ARHITEKTURA SISTEMA	17
6.2. TEHNOLOŠKI STACK	18
6.3. MODEL SLOJ I DOMENSKI ENTITETI	19
6.4. POINT-IN-TIME RECOVERY	20
6.5. DEMONSTRACIJA REPLIKACIJE	20
6.6. DEMONSTRACIJA FAILOVER SCENARIJA	22
6.7. DEMONSTRACIJA FAILBACK SCENARIJA.....	23
6.8. MERENJA I REZULTATI EKSPERIMENATA	25
6.9. ANALIZA REZULTATA.....	25
7. ZAKLJUČAK.....	27
LITERATURA	29

1. UVOD

U savremenim informacionim sistemima dostupnost podataka predstavlja jedan od ključnih zahteva koji se postavlja pred svaki sistem za upravljanje bazama podataka. Korisnici i aplikacije koje oslanjaju rad na podatke ne smeju biti suočeni sa situacijom u kojoj podaci nisu dostupni, bilo da je reč o privremenom kvaru hardvera, nedostupnosti mreže ili planiranom gašenju infrastrukture. Svaki period nedostupnosti sistema direktno se odražava na korisničko iskustvo, poslovne procese i, u kritičnim sistemima poput finansija ili zdravstva, može imati dalekosežne posledice. Zbog toga je pojam visoke dostupnosti, poznat i pod engleskim terminom *high availability*, postao centralni zahtev u projektovanju modernih distribuiranih sistema.

Tradicionalni sistemi za upravljanje relacionim bazama podataka dugo su se oslanjali na pristup koji podrazumeva jedan centralni server koji čuva sve podatke. Ovaj pristup, poznat kao *single point of failure* arhitektura, nosi ozbiljan rizik: ukoliko centralni server postane nedostupan iz bilo kog razloga, ceo sistem prestaje da funkcioniše. Rešenje ovog problema leži u replikaciji podataka, to jest u procesu kojim se isti podaci čuvaju na više fizičkih ili virtuelnih lokacija istovremeno. Na taj način, ukoliko jedan čvor sistema prestane da radi, drugi čvorovi mogu preuzeti njegovu ulogu bez prekida u radu sistema.

U kontekstu NoSQL baza podataka, MongoDB predstavlja jedan od najrasprostranjenijih sistema koji nativno podržava replikaciju kao osnovnu funkcionalnost. MongoDB je dokumentno orijentisana baza podataka otvorenog koda koja za razliku od relacionih sistema ne zahteva fiksnu šemu podataka, već omogućava fleksibilno čuvanje podataka u obliku dokumenata sličnih JSON formatu. Zahvaljujući svojoj arhitekturi i bogatom skupu funkcionalnosti, MongoDB je pronašao primenu u velikom broju različitih domena, od veb aplikacija i e-commerce platformi do analitičkih sistema i IoT rešenja.

Mehanizam replikacije u MongoDB sistemu realizuje se kroz koncept koji se naziva Replica Set. Replica Set predstavlja grupu MongoDB servera, od kojih jedan ima ulogu primarnog čvora (*Primary*), dok ostali imaju ulogu sekundarnih čvorova (*Secondary*). Primarni čvor prima sve operacije pisanja i replikuje promene ka sekundarnim čvorovima kroz mehanizam zasnovan na operacionom dnevniku, poznatom kao oplog. U slučaju da primarni čvor postane nedostupan, preostali čvorovi automatski pokreću proces izbora novog primarnog čvora, čime se osigurava kontinuitet rada sistema bez potrebe za ručnom intervencijom administratora.

Cilj ovog rada jeste da se na teorijskom i praktičnom nivou objasni način rada MongoDB Replica Set arhitekture, sa posebnim fokusom na mehanizme koji osiguravaju visoku dostupnost sistema. U radu su opisani principi replikacije podataka, heartbeat mehanizam za praćenje zdravlja čvorova, algoritam za izbor novog primarnog čvora, kao i koncepti kao što su *Read Preference* i *Write Concern* koji direktno utiču na ponašanje sistema u pogledu konzistentnosti i performansi. Pored teorijskog dela, u radu je prikazan i praktični deo koji obuhvata implementaciju Spring Boot aplikacije koja se povezuje na MongoDB Replica Set okruženje postavljeno u Docker kontejnerima. Kroz implementirane REST endpointe demonstrirani su ključni scenariji: merenje kašnjenja replikacije, razlika između čitanja sa primarnog i sekundarnog čvora, poređenje različitih nivoa *Write Concern*, kao i automatski failover i oporavak sistema. Rezultati eksperimenata su izmereni i analizirani kako bi se potvrdila teorijska razmatranja.

Rad je organizovan na sledeći način. U drugom poglavlju data je kraća analiza MongoDB sistema kao predstavnika NoSQL baza podataka, sa osvrtom na dokumentni model i osnovne karakteristike sistema. Treće poglavlje opisuje arhitekturu Replica Set sistema, ulogu pojedinih čvorova i mehanizam replikacije kroz oplog. Četvrto poglavlje posvećeno je mehanizmima visoke dostupnosti, uključujući failover proces, heartbeat protokol i election algoritam. U petom poglavlju razmatrane su prednosti i ograničenja Replica Set arhitekture. Šesto poglavlje, kao centralni deo rada, opisuje implementaciju praktičnog sistema, korišćene tehnologije, konfiguraciju okruženja i rezultate svih sprovedenih eksperimenata. Zaključna razmatranja i literatura navedeni su na kraju rada.

2. MongoDB kao NoSQL sistem

2.1. NoSQL baze podataka

Pojam NoSQL odnosi se na široku kategoriju sistema za upravljanje bazama podataka koji se razlikuju od tradicionalnog relacionog modela. Naziv sam po sebi ne znači da ovi sistemi ne koriste upitni jezik sličan SQL-u, već pre svega označava pristup koji nije zasnovan na striktnoj tabelarnoj strukturi i relacionim vezama između podataka. NoSQL sistemi nastali su kao odgovor na izazove koje su doneli masovni internet servisi, kod kojih su tradicionalne relacione baze podataka pokazivale ograničenja u pogledu skalabilnosti, fleksibilnosti šeme i brzine obrade velikih količina nestrukturiranih ili polustrukturiranih podataka.

NoSQL baze podataka mogu se podeliti u nekoliko osnovnih kategorija prema načinu na koji organizuju podatke. Dokumentne baze podataka, kao što je MongoDB, čuvaju podatke u obliku dokumenata koji mogu imati različitu strukturu. Baze podataka zasnovane na ključ-vrednost parovima, kao što su Redis i DynamoDB, čuvaju podatke u obliku jednostavnih parova bez složene strukture. Kolonarne baze podataka, poput Apache Cassandra, optimizovane su za rad sa velikim brojem kolona i analitičke upite. Graf baze podataka, kao što je Neo4j, specijalizovane su za čuvanje i pretraživanje podataka koji su po prirodi organizovani kao grafikoni sa čvorovima i vezama između njih.

Zajednička karakteristika većine NoSQL sistema jeste horizontalna skalabilnost, to jest mogućnost dodavanja novih servera u sistem kako bi se rasporedilo opterećenje, za razliku od vertikalne skalabilnosti koja podrazumeva nadogradnju postojećeg servera. Pored toga, NoSQL sistemi često žrtvuju određeni stepen konzistentnosti podataka u korist veće dostupnosti i tolerancije na particije mreže, što je formalizovano kroz CAP teorem koji tvrdi da distribuirani sistem ne može istovremeno garantovati konzistentnost, dostupnost i toleranciju na mrežne particije.

2.2. Karakteristike MongoDB sistema

MongoDB je dokumentno orijentisana baza podataka otvorenog koda koju je razvila kompanija MongoDB Inc., a čija je prva verzija objavljena 2009. godine. Sistem je napisan u programskom jeziku C++ i dostupan je za sve popularnije operativne sisteme. MongoDB se danas redovno nalazi među najkorišćenijim bazama podataka u industriji i posebno je rasprostranjen u veb i mobilnim aplikacijama, kao i u sistemima koji rade sa podacima čija se struktura često menja.

Osnovna karakteristika koja razlikuje MongoDB od relacionih sistema jeste odsustvo fiksne šeme. U relacionoj bazi podataka svaka tabela ima unapred definisane kolone i tipove podataka, a svaki red mora da odgovara toj definiciji. MongoDB ne nameće ovakvo ograničenje: svaki dokument u kolekciji može imati drugačiji skup polja, a polja mogu sadržavati vrednosti različitih tipova. Ova osobina donosi fleksibilnost koja je posebno korisna u ranim fazama razvoja aplikacije kada se model podataka često menja, ali zahteva i pažljiv pristup projektovanju kako bi se izbegla nekonzistentnost podataka na nivou aplikacione logike.

MongoDB podržava bogat upitni jezik koji omogućava filtriranje, sortiranje, grupisanje i agregaciju podataka. Pored toga, sistem podržava indeksiranje po bilo kom polju dokumenta, uključujući i polja ugniježđenih dokumenata i nizova, što omogućava efikasno pretraživanje i bez linearnog skeniranja cele kolekcije. Transakcije sa ACID svojstvima podržane su od verzije 4.0, a od verzije 4.2 omogućene su i distribuirane transakcije koje obuhvataju više čvorova u Replica Set okruženju.

2.3. Dokumentni model i BSON format

Osnovna jedinica podataka u MongoDB sistemu jeste dokument. Dokument je struktura podataka koja se sastoji od polja i vrednosti, konceptualno slična JSON objektu. Svaki dokument mora imati polje **id** koje predstavlja jedinstveni identifikator dokumenta unutar kolekcije. Ukoliko ovo polje nije eksplicitno navedeno prilikom umetanja dokumenta, MongoDB ga automatski generiše kao vrednost tipa **ObjectId**.

Primer jednostavnog dokumenta koji predstavlja korisnika može izgledati ovako:

```
{
  "_id": "a3f1c2jcn7ffd373bf",
  "ime": "Andrea",
  "prezime": "Valčić",
  "email": "andrea.valcic@elfak.rs",
  "adresa": {
    "grad": "Niš",
    "postanskiBroj": "18000"
  },
  "tagovi": ["student", "aktivni"]
}
```

Slika 1. Primer MongoDB dokumenta sa ugniježđenim poljem i nizom vrednosti

Ovaj primer ilustruje dve važne osobine dokumentnog modela. Prvo, polje **adresa** sadrži ugniježđeni dokument, što omogućava prirodno modelovanje hijerarhijskih podataka bez potrebe za join operacijom kakva je uobičajena u relacionim bazama. Drugo, polje **tagovi** sadrži niz vrednosti, što je direktno podržano bez potrebe za posebnom tabelom veza kakva bi bila neophodna u relacionom modelu.

Iako su dokumenti konceptualno slični JSON-u, MongoDB interno koristi binarni format koji se naziva BSON (*Binary JSON*). BSON format proširuje JSON dodavanjem tipova podataka koji nisu podržani u standardnom JSON-u, kao što su datum i vreme (**Date**), binarni podaci (**BinData**), regularni izrazi i tip **ObjectId** koji se koristi za generisanje jedinstvenih identifikatora. Binarno kodiranje BSON formata omogućava efikasnije čuvanje i pretraživanje podataka u poređenju sa tekstualnim JSON formatom, kao i direktno parsiranje bez potrebe za konverzijom stringa u strukturirani objekat.

Dokumenti su organizovani u kolekcije, koje su analogne tabelama u relacionim bazama podataka. Međutim, za razliku od tabela, kolekcija ne nameće nikakvu šemu dokumentima koje sadrži. Više kolekcija grupišu se unutar jedne baze podataka, a jedan MongoDB server može da sadrži više baza podataka istovremeno.

2.4. Prednosti i ograničenja MongoDB sistema

Među glavnim prednostima MongoDB sistema ističe se fleksibilnost šeme koja olakšava razvoj i evoluiranje aplikacija tokom vremena. Dokumentni model je u mnogim slučajevima prirodniji za reprezentaciju podataka koji se koriste u objektno orijentisanim aplikacijama, jer omogućava direktno preslikavanje objekata na dokumente bez kompleksnog mapiranja kakvo zahteva relacioni model. Horizontalna skalabilnost kroz *sharding* mehanizam omogućava raspoređivanje podataka na više servera, a nativna podrška za replikaciju kroz Replica Set obezbeđuje visoku dostupnost sistema bez dodatnih alata ili konfiguracije.

Sa druge strane, MongoDB nosi i određena ograničenja. Fleksibilnost šeme može biti i mana ukoliko se ne primenjuju konzistentna pravila na nivou aplikacije, jer MongoDB ne može garantovati strukturu podataka bez eksplicitno definisanih validacionih pravila. Do verzije 4.0 MongoDB nije podržavao višedokumentne transakcije, što ga je u određenim domenima činilo neprikladnim izborom. Pored toga, sistem nije optimalan za slučajeve koji zahtevaju složene join operacije između različitih kolekcija, jer MongoDB ne podržava relacione veze na nivou baze podataka na način koji je ugrađen u relacione sisteme.

Može se zaključiti da MongoDB kao NoSQL sistem nudi skup karakteristika koji ga čini prikladnim izborom za određenu klasu problema, posebno u scenarijima gde su fleksibilnost, skalabilnost i visoka dostupnost prioriteta. Upravo mehanizam Replica Set, koji je centralna tema ovog rada, direktno adresira zahtev za visokom dostupnošću i predstavlja jednu od ključnih arhitektonskih odluka u projektovanju MongoDB sistema za produkcijsko okruženje.

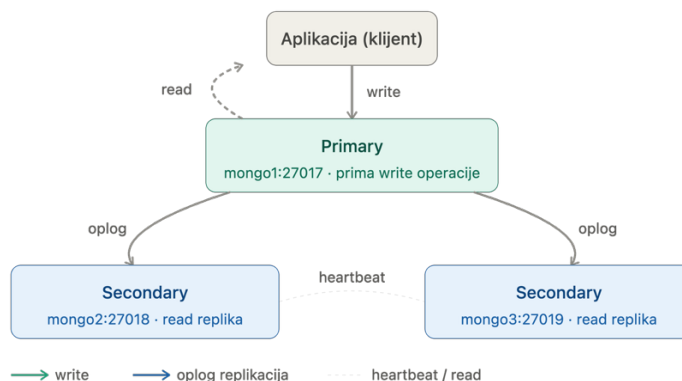
3. Replica Set arhitektura

3.1. Osnove Replica Set koncepta

Replica Set u MongoDB sistemu predstavlja grupu mongod procesa koji zajedno čuvaju isti skup podataka. Osnovna svrha ovog mehanizma jeste obezbeđivanje redundantnosti podataka i visoke dostupnosti sistema. Umesto da se podaci čuvaju na jednom serveru, Replica Set distribuira kopije podataka na više čvorova, čime se eliminiše *single point of failure* problem koji karakteriše sisteme sa jednim serverom. Svaki čvor u Replica Set grupi u svakom trenutku zna za postojanje ostalih članova i aktivno prati njihovo zdravstveno stanje.

Replica Set se sastoji od najmanje tri člana kako bi se osiguralo ispravno funkcionisanje algoritma za izbor primarnog čvora. Sa dva člana, u slučaju nedostupnosti jednog, preostali čvor ne može da formira većinu i ne može da postane primarni, čime sistem praktično ostaje bez mogućnosti pisanja. Sa tri ili više članova, preostala dva čvora mogu formirati većinu i izabrati novog primarnog. Preporučeni broj članova u produkcijskom okruženju jeste neparan broj, jer algoritam za izbor zahteva striktnu većinu glasova.

3.2. Uloge čvorova u Replica Set sistemu



Slika 2. Arhitektura MongoDB Replica Set sistema sa tri čvora

Svaki čvor u Replica Set sistemu ima dodeljenu ulogu koja određuje njegovo ponašanje u pogledu prihvatanja operacija čitanja i pisanja.

Primarni čvor (*Primary*) jedini prihvata operacije pisanja u normalnom stanju sistema. Sve izmene podataka, bilo da se radi o umetanju novih dokumenata, ažuriranju postojećih ili brisanju, moraju biti upućene primarnom čvoru. Nakon što primarni čvor primeni operaciju pisanja, on beleži tu izmenu u operacioni dnevnik poznat kao oplog, odakle je sekundarni čvorovi preuzimaju i primenjuju na sopstvenim kopijama podataka. U bilo kom trenutku u okviru jednog Replica Set sistema može postojati samo jedan primarni čvor. Praktični deo ovog rada potvrdio je ovo ponašanje: u svim merenim stanjima sistema jedan i samo jedan čvor nosio je oznaku PRIMARY.

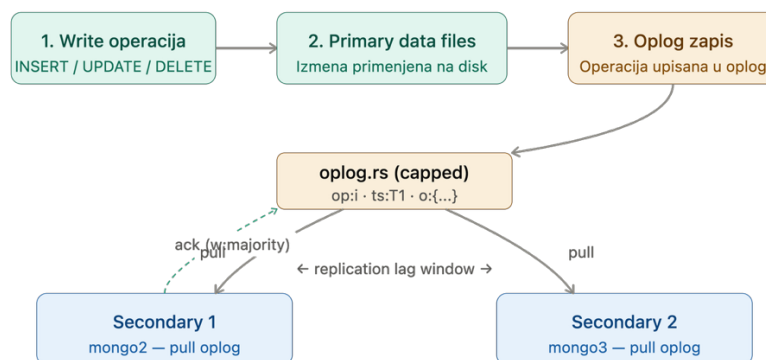
Sekundarni čvorovi (*Secondary*) održavaju kopije podataka koje su replikovane sa primarnog čvora. Sekundarni čvorovi kontinuirano čitaju oplog primarnog čvora i primenjuju iste operacije na sopstvenim skupovima podataka, čime ostaju sinhronizovani sa primarnim. Po podrazumevanom podešavanju, sekundarni čvorovi ne prihvataju operacije pisanja, ali mogu da odgovaraju na upite čitanja ukoliko je to eksplicitno konfigurisano kroz Read Preference mehanizam, koji je detaljno opisan u poglavlju 4. U slučaju nedostupnosti primarnog čvora, sekundarni čvorovi biraju jednog između sebe koji preuzima ulogu primarnog.

Arbiter je specijalni tip člana koji ne čuva kopiju podataka, već isključivo učestvuje u procesu glasanja pri izboru novog primarnog čvora. Arbiter se koristi u situacijama kada je potrebno obezbediti neparan broj glasova bez troška dodatnog servera koji bi čuvao punu kopiju podataka. Na primer, u konfiguraciji sa jednim primarnim i jednim sekundarnim čvorom, dodavanje arbitera obezbeđuje da se u slučaju otkaza primarnog može formirati većina i izabrati novi primarni. Treba napomenuti da arbiter nije pogodan za sve produkcijske scenarije jer ne doprinosi redundantnosti podataka.

3.3. Mehanizam replikacije i oplog

Replikacija u MongoDB Replica Set sistemu zasniva se na asinhronom prenosu podataka putem operacionog dnevnika, koji se u MongoDB terminologiji naziva oplog (*operations log*). Oplog je posebna kolekcija tipa *capped collection* koja se nalazi u sistemskoj bazi podataka **local** na svakom članu Replica Set grupe. Kolekcija tipa *capped* ima fiksni maksimalni kapacitet, a kada se taj kapacitet dostigne, najstariji unosi se automatski prepisuju novim, što znači da oplog čuva samo određeni prozor istorije operacija.

Svaki put kada primarni čvor izvrši operaciju pisanja, ta operacija se upisuje u oplog u obliku idempotentnog zapisa. Idempotentnost znači da se ista operacija može primeniti više puta, a rezultat će uvek biti isti, što je važna osobina jer omogućava pouzdano ponavljanje operacija u slučaju prekida replikacije. Na primer, operacija koja postavlja vrednost polja na konkretnu vrednost idempotentna je, dok operacija koja inkrementira vrednost za jedan nije idempotentna u svom izvornom obliku, pa MongoDB oplog takve operacije konvertuje u ekvivalentne idempotentne zapise.



Slika 3. Tok replikacije podataka kroz oplog mehanizam

Sekundarni čvorovi kontinuirano prate oplog primarnog i kopiraju nove unose. Ovaj proces poznat je kao oplog tailing i odvija se u pozadinskom procesu na svakom sekundarnom čvoru. Nakon što sekundarni čvor preuzme novi unos iz oploga, primenjuje odgovarajuću operaciju na sopstvenoj kopiji podataka. Budući da se ovaj proces odvija asinhrono, uvek postoji određeni vremenski razmak između trenutka kada primarni čvor primeni operaciju i trenutka kada sekundarni čvor primeni istu operaciju. Ovaj razmak naziva se kašnjenje replikacije (*replication lag*) i njegova veličina zavisi od mrežnih uslova, opterećenja sistema i raspoloživih resursa na sekundarnim čvorovima.

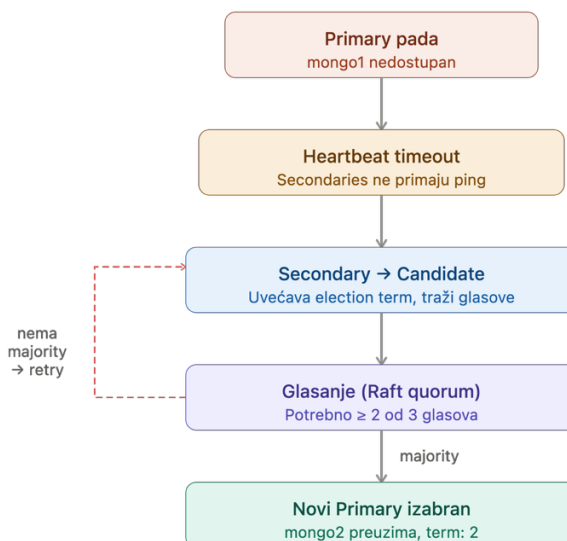
Eksperiment sproveden u praktičnom delu rada direktno meri ovo kašnjenje. Rezultati su pokazali da je u lokalnom Docker okruženju kašnjenje replikacije iznosilo svega 15 ms, pri čemu je dokument bio vidljiv na sekundarnom čvoru već nakon prvog pokušaja čitanja. Ovakav rezultat očekivan je u okruženju gde svi čvorovi komuniciraju unutar iste mašine bez mrežnog kašnjenja, ali u produkcijskim okruženjima sa geografski raspoređenim čvorovima ovo kašnjenje može biti značajno veće.

3.4. Heartbeat mehanizam i praćenje zdravlja čvorova

Svaki čvor u Replica Set grupi periodično šalje heartbeat poruke svim ostalim članovima kako bi proverio da li su dostupni i u kakvom su stanju. Podrazumevani interval između heartbeat poruka iznosi dve sekunde. Ukoliko čvor ne primi odgovor od drugog člana unutar definisanog vremenskog ograničenja, koje podrazumevano iznosi 10 sekundi i može se konfigurisati parametrom **electionTimeoutMillis**, taj član se smatra nedostupnim.

Heartbeat poruke nose informacije o trenutnom stanju čvora koji ih šalje, uključujući identifikator replikacionog termina (*term*), poslednju primenjenu optime vrednost i trenutnu ulogu čvora. Ove informacije koriste se kako za praćenje zdravlja sistema tako i za donošenje odluke o potrebi pokretanja procesa izbora novog primarnog čvora.

3.5. Proces izbora primarnog čvora (Election)



Slika 4. Dijagram toka procesa izbora novog primarnog čvora

Kada primarni čvor postane nedostupan, bilo zbog hardverskog kvara, mrežnog problema ili planiranog gašenja, preostali sekundarni čvorovi pokreću proces izbora novog primarnog. Ovaj proces zasnovan je na protokolu sličnom Raft algoritmu za distribuirani konsenzus i odvija se u nekoliko faza.

U prvoj fazi, sekundarni čvor koji primeti nedostupnost primarnog čeka određeni slučajni vremenski interval pre nego što kandiduje sebe za novog primarnog. Slučajni vremenski interval uveden je kako bi se smanjila verovatnoća istovremenog kandidovanja više čvorova, što bi moglo dovesti do situacije u kojoj nijedan čvor ne uspeva da dobije dovoljan broj glasova. Čvor koji se kandiduje povećava vrednost termina za jedan i šalje zahtev za glasanjem (*requestVote*) svim ostalim dostupnim članovima. .

Svaki čvor koji primi zahtev za glasanjem razmatra nekoliko uslova pre nego što da glas. Kandidat mora imati vrednost oploga koja je jednaka ili novija od oploga čvora koji razmatra zahtev, što osigurava da novi primarni čvor poseduje najaktuelnije podatke. Ukoliko su ovi uslovi ispunjeni i čvor još nije glasao u trenutnom terminu, daje glas kandidatu. Kandidat koji dobije glasove od striktno većine članova Replica Set grupe (minimalno $\lfloor n/2 \rfloor + 1$ glasova, gde je n ukupan broj članova) postaje novi primarni čvor.

Ovaj mehanizam osigurava da u svakom trenutku postoji najviše jedan primarni čvor i da novi primarni uvek poseduje sve potvrđene operacije pisanja. Ukoliko nijedan kandidat ne dobije dovoljan broj glasova, proces se ponavlja sa novim terminom sve dok se ne izabere novi primarni ili dok ne prođe definisani vremenski limit.

Praktični eksperiment opisuje izuzetno brz proces izbora: nakon gašenja primarnog čvora `mongo1:27017`, novi primarni čvor **mongo2:27018** izabran je za svega 506 ms. Ovaj rezultat pokazuje da MongoDB Replica Set u optimalnim uslovima može obnoviti dostupnost pisanja za manje od jedne sekunde, što ga čini pogodnim za sisteme koji postavljaju visoke zahteve u pogledu neprekidnosti rada.

3.6. Konfiguracija i prioriteta čvorova

Svaki član Replica Set grupe može imati definisan prioritet koji utiče na to koji čvor ima prednost pri kandidovanju za primarnog. Prioritet se izražava kao decimalni broj u opsegu od 0 do 1000, a podrazumevana vrednost je 1. Čvorovi sa višim prioritetom imaju prednost pri glasanju, dok čvorovi sa prioritetom 0 ne mogu postati primarni, ali učestvuju u glasanju i mogu čuvati kopije podataka. Ova mogućnost korisna je u okruženjima gde određeni serveri imaju bolje hardverske karakteristike i trebalo bi da nose ulogu primarnog u normalnim uslovima.

U praktičnom delu rada, Replica Set je konfigurisan sa tri čvora pri čemu je čvoru **mongo1** dodeljen prioritet 2, a čvorovima **mongo2** i **mongo3** prioritet 1. Ova konfiguracija osigurala je da **mongo1** postane inicijalni primarni čvor, ali nije sprečila **mongo2** da preuzme ulogu primarnog tokom failover eksperimenta. Nakon oporavka i ponovnog priključivanja **mongo1** čvora sistemu, on je preuzeo ulogu primarnog zahvaljujući višem prioritetu, što se može videti iz rezultata endpoint poziva `/api/replica-set/status` nakon završetka failback eksperimenta: vrednost polja **term** povećana je na 3, a **mongo1** je ponovo nosio oznaku PRIMARY.

4. Mehanizmi visoke dostupnosti

4.1. Pojam visoke dostupnosti

Visoka dostupnost (*High Availability*, skraćeno HA) označava svojstvo sistema da ostane operativan i dostupan korisnicima tokom što dužeg vremenskog perioda, čak i u prisustvu kvarova pojedinih komponenti. U kontekstu sistema za upravljanje bazama podataka, visoka dostupnost podrazumeva da sistem nastavlja da prihvata i obrađuje upite čitanja i pisanja bez dugih perioda nedostupnosti, bez obzira na to da li je uzrok problema hardverski kvar, mrežni incident, softverska greška ili planirano održavanje infrastrukture.

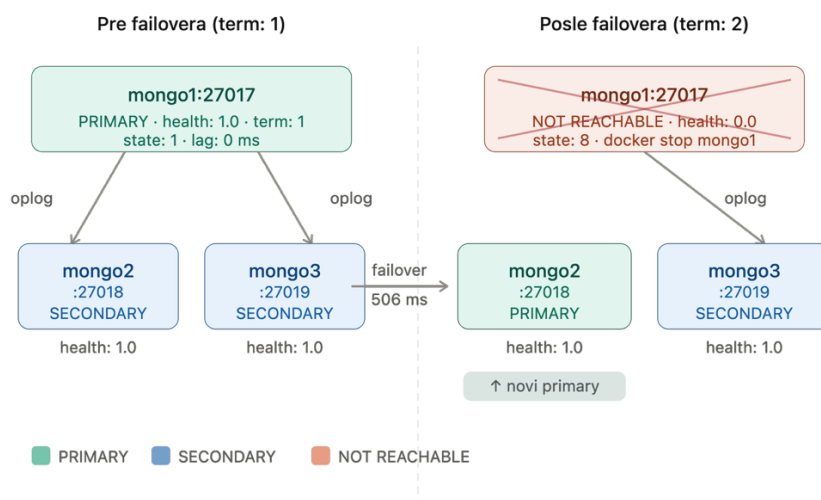
Dostupnost sistema obično se izražava kao procenat vremena u toku kojeg je sistem operativan u posmatranom periodu. Standard koji se često koristi u industriji jeste takozvana dostupnost od "pet devetki", što odgovara vrednosti od 99,999%, a to znači da je sistem nedostupan ukupno manje od pet i po minuta godišnje. Postizanje ovako visokog stepena dostupnosti nije moguće sa arhitekturom zasnovanom na jednom serveru, jer svaki realan sistem podložen je povremenim kvarovima. Jedini način da se ovakvi ciljevi ostvare jeste uvođenje redundantnosti na svim nivoima sistema, uključujući i nivo baze podataka.

MongoDB Replica Set arhitektura direktno adresira ovaj zahtev kroz kombinaciju više mehanizama koji zajedno osiguravaju da sistem može da nastavi rad čak i kada jedan ili više čvorova postane nedostupno.

4.2. Automatski failover

Failover označava proces u kojem sistem automatski prebacuje odgovornost za obradu zahteva sa nedostupne komponente na drugu, funkcionalnu komponentu. U kontekstu MongoDB Replica Set sistema, failover se odvija kada primarni čvor postane nedostupan, a preostali sekundarni čvorovi automatski biraju novog primarnog bez potrebe za intervencijom administratora.

Proces automatskog failovera u MongoDB sistemu može se opisati kroz nekoliko faza. Kada primarni čvor prestane da odgovara na heartbeat poruke u okviru definisanog vremenskog ograničenja (**`electionTimeoutMillis`**, podrazumevano 10 sekundi), sekundarni čvorovi zaključuju da je primarni nedostupan i pokreću process izbora. Tokom trajanja ovog procesa, sistem privremeno nije u mogućnosti da prihvata operacije pisanja, jer ne postoji aktivan primarni čvor. Nakon što novi primarni bude izabran, sistem nastavlja normalan rad, a klijentske aplikacije koje koriste MongoDB drajver automatski se preusmeravaju na novi primarni čvor bez potrebe za promenom konfiguracije.



Slika 5. Stanje Replica Set sistema pre i nakon failover procesa

Važno je naglasiti da automatski failover ne garantuje nultu nedostupnost. Tokom perioda od trenutka kada primarni postane nedostupan do trenutka kada novi primarni bude izabran i konfigurisan, sistem neće prihvatati operacije pisanja. Dužina ovog perioda zavisi od konfiguracije **electionTimeoutMillis** parametra i mrežnih uslova. U praktičnom eksperimentu sprovedenom u okviru ovog rada, ovaj period iznosio je svega 506 ms, što je izuzetno kratko vreme i pokazuje da MongoDB u optimalnim uslovima može obezbediti gotovo neprekidan rad.

4.3. Read Preference

Read Preference je mehanizam koji definiše sa kojeg člana Replica Set grupe klijentska aplikacija čita podatke. MongoDB drajver podržava nekoliko režima čitanja koji nude različite kompromise između konzistentnosti podataka i performansi sistema.

Režim **primary** usmerava sve operacije čitanja isključivo na primarni čvor. Ovaj režim garantuje da će aplikacija uvek čitati najnovije potvrđene podatke, ali sve operacije čitanja opterećuju isti čvor koji obrađuje i operacije pisanja. Ovo je podrazumevani režim i preporučuje se za scenarije u kojima je konzistentnost podataka kritična.

Režim **primaryPreferred** usmerava operacije čitanja na primarni čvor kada je dostupan, a na sekundarni u slučaju nedostupnosti primarnog. Ovaj režim koristan je kao rezervno rešenje koje ne žrtvuje konzistentnost u normalnom radu, ali i dalje funkcioniše tokom kratkih perioda failovera.

Režim **secondary** usmerava sve operacije čitanja isključivo na sekundarne čvorove. Budući da sekundarni čvorovi mogu biti blago zaostali za primarnim zbog asinhronne replikacije, ovaj režim može rezultovati čitanjem zastarelih podataka. Prihvatljiv je za scenarije u kojima blaga zaostalost nije problem, kao što su analitički upiti, generisanje izveštaja ili prikaz podataka koji se ne menjaju često.

Režim **secondaryPreferred** usmerava operacije čitanja na sekundarne čvorove kada su dostupni, a na primarni kada nijedan sekundarni čvor nije dostupan. Ovaj režim je kompromis između opterećenja primarnog čvora i konzistentnosti podataka.

Režim **nearest** usmerava operacije čitanja na onaj čvor koji ima najmanji mrežni kašnjenje (*latency*) u odnosu na klijenta, bez obzira na to da li je reč o primarnom ili sekundarnom čvoru. Koristan je u geografski distribuiranim okruženjima gde je minimizacija latencije prioritet.

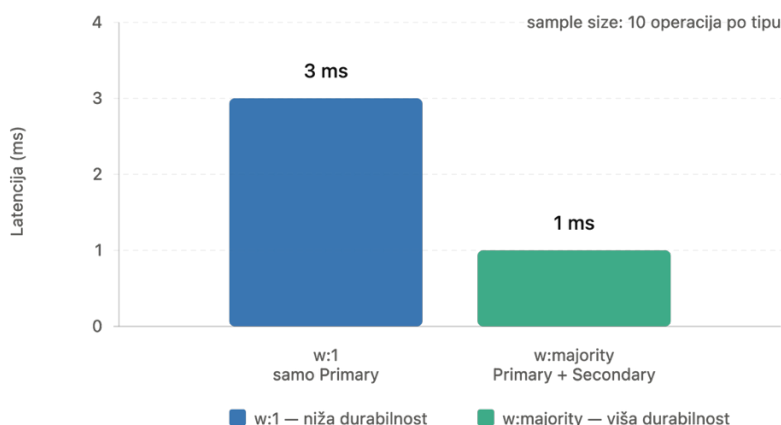
U praktičnoj implementaciji opisanoj u ovom radu, eksperiment sa Read Preference demonstrirao je razliku između čitanja sa primarnog i sekundarnog čvora. Rezultati su pokazali identičnu latenciju od 2 ms za oba režima, što je očekivano u lokalnom okruženju gde svi čvorovi komuniciraju unutar iste mašine. U produkcijskim okruženjima sa geografski raspoređenim čvorovima, razlika u latenciji između primarnog i sekundarnog čvora može biti značajna i predstavlja jedan od ključnih argumenata za korišćenje **secondaryPreferred** režima.

4.4. Write Concern

Write Concern definiše nivo potvrde koji MongoDB mora da dobije od članova Replica Set grupe pre nego što operaciju pisanja označi kao uspešnu i vrati odgovor klijentu. Ovaj parametar direktno utiče na kompromis između latencije pisanja i trajnosti podataka.

Nivo **w:1** znači da samo primarni čvor mora da potvrdi prijem i primenu operacije pisanja. Ovo je najbrži mogući nivo jer ne čeka na replikaciju ka sekundarnim čvorovima. Međutim, ukoliko primarni čvor otkáže neposredno nakon što je potvrdio operaciju, a pre nego što je ta operacija replicirana na sekundarne čvorove, postoji mogućnost gubitka podataka. Ovaj nivo prihvatljiv je za scenarije gde je brzina pisanja prioritet, a mogući gubitak određenih operacija nije kritičan.

Nivo **w:majority** znači da mora da odgovori striktna većina članova Replica Set grupe pre nego što se klijentu vrati potvrda o uspešnoj operaciji pisanja. Ovo garantuje da će podaci preživeti failover primarnog čvora, jer su već prisutni na dovoljnom broju čvorova da bi novi primarni mogao biti izabran bez gubitka tih podataka. Cena ovog nivoa trajnosti jeste veća latencija, jer operacija pisanja mora sačekati na odgovor od sekundarnih čvorova.



Slika 6. Poređenje latencije pisanja za w:1 i w:majority nivo

Eksperiment sproveden u praktičnom delu rada pokazao je zanimljiv rezultat: prosečna latencija za **w:1** iznosila je 3 ms, dok je prosečna latencija za **w:majority** iznosila 1 ms. Ovaj naizgled neočekivani rezultat, gde je stroži nivo trajnosti pokazao nižu latenciju, posledica je specifičnosti lokalnog okruženja u kome svi čvorovi komuniciraju unutar iste fizičke mašine bez mrežnog kašnjenja. U produkcijskim okruženjima sa čvorovima raspoređenim na različitim serverima ili geografskim lokacijama, **w:majority** uvek rezultuje višom latencijom od **w:1**, jer zahteva dodatni mrežni prenos ka sekundarnim čvorovima i natrag.

Pored **w:1** i **w:majority**, MongoDB podržava i precizno definisanje broja čvorova koji moraju potvrditi operaciju, kao što je **w:2** ili **w:3**, što pruža dodatnu fleksibilnost u konfiguraciji sistema prema konkretnim zahtevima.

4.5. Read Concern

Read Concern je mehanizam koji definiše nivo konzistentnosti podataka koji se garantuje pri čitanju iz MongoDB sistema. Ovaj parametar komplementaran je sa *Write Concern* mehanizmom i zajedno s njim određuje ukupne garancije konzistentnosti koje sistem pruža.

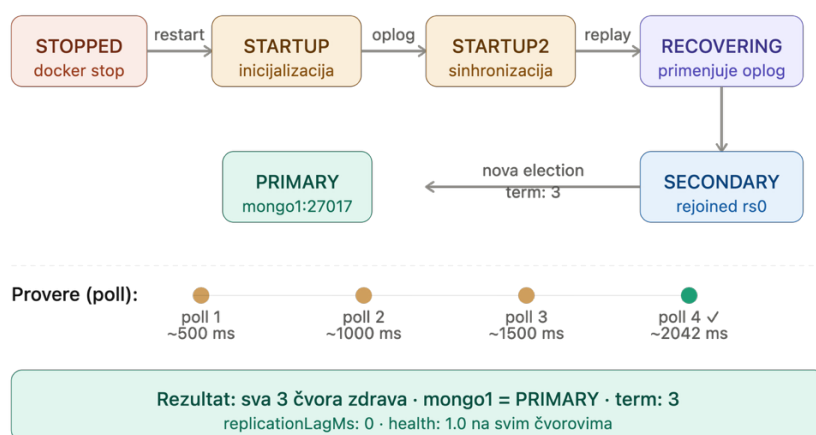
Nivo **local** vraća podatke koji su lokalno prisutni na čvoru u trenutku čitanja, bez garancije da su ti podaci potvrđeni od strane većine članova. Ovo je podrazumevani nivo i pruža najbrže čitanje, ali može vratiti podatke koji će biti poništeni (*rollback*) u slučaju failovera.

Nivo **majority** vraća samo one podatke koji su potvrđeni od strane većine članova Replica Set grupe. Ovaj nivo garantuje da pročitani podaci neće biti poništeni, ali može rezultovati čitanjem blago zastarelih podataka jer sistema čeka na potvrdu replikacije.

Nivo **linearizable** pruža najjače garancije konzistentnosti: garantuje da čitanje reflektuje sve operacije pisanja koje su potvrđene pre početka čitanja. Ovo je korisno za scenarije koji zahtevaju striktnu linearizabilnost, ali dolazi po cenu značajno veće latencije čitanja.

4.6. Oporavak sistema nakon failovera

Nakon što novi primarni čvor bude izabran i sistem nastavi normalan rad, prethodni primarni čvor može biti ponovo pokrenut i pridružen Replica Set grupi. Proces ponovnog pridruživanja odvija se automatski i ne zahteva ručnu konfiguraciju. Kada se prethodni primarni čvor ponovo pokrene, on kontaktira trenutnog primarnog i identifikuje koje oplog unose propustio tokom perioda nedostupnosti. Ukoliko su propuštene operacije i dalje prisutne u oplog-u trenutnog primarnog, čvor ih preuzima i primenjuje u procesu koji se naziva inkrementalna sinhronizacija. Ukoliko je čvor bio nedostupan dovoljno dugo da propuštene operacije više nisu u oplog-u, pokreće se proces inicijalne sinhronizacije (*initial sync*) koji podrazumeva potpuno preuzimanje svih podataka sa trenutnog primarnog.



Slika 7. Proces oporavka i ponovnog pridruživanja čvora Replica Set grupi

Tokom procesa oporavka, čvor prolazi kroz nekoliko stanja koja su vidljiva u **rs.status()** izlazu: **STARTUP**, zatim **STARTUP2** dok traje inicijalna sinhronizacija ili preuzimanje oploga, potom **RECOVERING** dok se primenjuju zaostale operacije, i konačno **SECONDARY** kada je sinhronizacija završena. Praktični eksperiment pokazao je da je čvor **mongo1:27017** prošao ovaj ceo proces za svega 2042 ms, pri čemu je potreban oporavak bio minimalan zbog kratke nedostupnosti tokom failover eksperimenta.

Važna karakteristika oporavka jeste da se prethodni primarni čvor ne vraća automatski u ulogu primarnog, čak i ako ima viši prioritet. Trenutni primarni čvor ostaje na toj poziciji sve dok ne dođe do novog failovera ili dok administrator eksplicitno ne pokrene **rs.stepDown()** komandu. Ovo ponašanje može se promeniti konfiguracijom prioriteta, ali podrazumevana vrednost sprečava nepotrebne prelaskе uloge koje bi uzrokovale kratke periode nedostupnosti pisanja.

5. Prednosti i ograničenja Replica Set arhitekture

5.1. Prednosti

MongoDB Replica Set arhitektura donosi niz značajnih prednosti koje je čine pogodnom za sisteme koji postavljaju visoke zahteve u pogledu dostupnosti i pouzdanosti podataka.

Automatska visoka dostupnost predstavlja najvažniju prednost Replica Set arhitekture. Za razliku od sistema zasnovanih na jednom serveru, Replica Set obezbeđuje da kvar jednog čvora ne uzrokuje nedostupnost celog sistema. Proces automatskog failovera odvija se bez potrebe za ručnom intervencijom, a klijentske aplikacije koje koriste MongoDB drayver automatski se preusmeravaju na novi primarni čvor. Praktični eksperiment pokazao je da ovaj proces može biti završen za manje od jedne sekunde u optimalnim uslovima.

Redundantnost podataka osigurava da svaka operacija pisanja koja je potvrđena sa nivoom **w:majority** postoji na više fizičkih lokacija istovremeno. Čak i u slučaju potpunog fizičkog kvara jednog servera, podaci ostaju dostupni na preostalim čvorovima. Ovo pruža zaštitu ne samo od softverskih grešaka već i od hardverskih kvarova koji bi u arhitekturi sa jednim serverom uzrokovali trajni gubitak podataka.

Skalabilnost čitanja može se postići distribucijom operacija čitanja na sekundarne čvorove kroz mehanizam *Read Preference*. U sistemima sa dominantnim opterećenjem čitanja, kao što su platforme za prikaz sadržaja ili analitički sistemi, ovo omogućava horizontalno raspoređivanje opterećenja bez potrebe za dodatnom infrastrukturom. Dodavanjem novih sekundarnih čvorova kapacitet čitanja sistema može se linearno povećavati.

Fleksibilna konfiguracija omogućava prilagođavanje Replica Set sistema konkretnim potrebama. Parametri kao što su **Write Concern**, **Read Preference**, prioriteti čvorova i vremenski limiti za election mogu se podešavati prema zahtevima aplikacije. Ova fleksibilnost pruža mogućnost preciznog definisanja kompromisa između performansi, konzistentnosti i dostupnosti.

Transparentnost za aplikaciju znači da MongoDB drajver interno upravlja otkrivanjem topologije Replica Set grupe, praćenjem dostupnosti čvorova i preusmeravanjem operacija. Aplikativni kod ne mora eksplicitno da rukuje logikom preusmeravanja ili oporavka od grešaka, što smanjuje složenost razvoja i mogućnost grešaka u aplikacionoj logici.

5.2. Ograničenja

Pored prednosti, Replica Set arhitektura nosi i određena ograničenja koja treba uzeti u obzir prilikom projektovanja sistema.

Kašnjenje replikacije je inherentno ograničenje asinhronog modela replikacije koji MongoDB koristi. Budući da se replikacija odvija asinhrono, uvek postoji vremenski razmak između trenutka kada primarni čvor primeni operaciju i trenutka kada sekundarni čvorovi primene istu operaciju. Tokom ovog perioda, čitanje sa sekundarnih čvorova može vratiti zastarele podatke. U distribuiranim okruženjima sa mrežnim kašnjenjem ovo kašnjenje može biti i do nekoliko stotina milisekundi, što može biti neprihvatljivo za aplikacije koje zahtevaju striktnu konzistentnost podataka.

Ograničena horizontalna skalabilnost pisanja predstavlja strukturno ograničenje Replica Set arhitekture. Sve operacije pisanja moraju proći kroz jedan primarni čvor, što znači da kapacitet pisanja sistema nije moguće povećati dodavanjem novih čvorova u Replica Set grupu. Za sisteme sa intenzivnim opterećenjem pisanja jedino rešenje je vertikalna nadogradnja primarnog čvora ili prelazak na sharding arhitekturu koja distribuira podatke i operacije pisanja na više particija.

Složenija administracija u poređenju sa sistemom zasnovanim na jednom serveru podrazumeva potrebu za praćenjem zdravlja svih čvorova, upravljanjem oplog kapacitetom, konfiguracijom mrežne komunikacije između čvorova i planiranjem kapaciteta. Administracija postaje dodatno složena u okruženjima sa geografski raspoređenim čvorovima, gde mrežni uslovi i latencija direktno utiču na ponašanje sistema.

Mogućnost rollback operacija postoji u scenariju gde primarni čvor prihvati operacije pisanja sa nivoom w:l, ali otkáže pre nego što te operacije budu replicirane na sekundarne čvorove. Kada se takav čvor ponovo pridruži sistemu, operacije koje nisu bile replicirane moraju biti poništene kako bi se čvor doveo u konzistentno stanje sa ostatkom Replica Set grupe. Ove poništene operacije smeštaju se u posebnu rollback datoteku i zahtevaju manuelnu intervenciju administratora ukoliko je potrebno sačuvati te podatke.

Povećani hardverski zahtevi su direktna posledica redundantnosti. Minimalna Replica Set konfiguracija sa tri čvora zahteva trostruko više resursa u poređenju sa sistemom zasnovanim na jednom serveru, ukoliko se koriste identični serveri. Iako je moguće koristiti sekundarne čvorove sa slabijim hardverom, svaki čvor mora imati dovoljno resursa da samostalno obradi sve operacije čitanja u slučaju preuzimanja uloge primarnog.

5.3. Kada koristiti Replica Set arhitekturu

Na osnovu analiziranih prednosti i ograničenja može se zaključiti da je Replica Set arhitektura pogodna za sisteme kod kojih je visoka dostupnost kritičan zahtev, kod kojih opterećenje čitanja dominira nad opterećenjem pisanja, i kod kojih je kapacitet jednog servera dovoljan za prihvatanje svih operacija pisanja. Tipični primeri su veb aplikacije, platforme za upravljanje sadržajem, mobilni backend servisi i sistemi za praćenje u realnom vremenu.

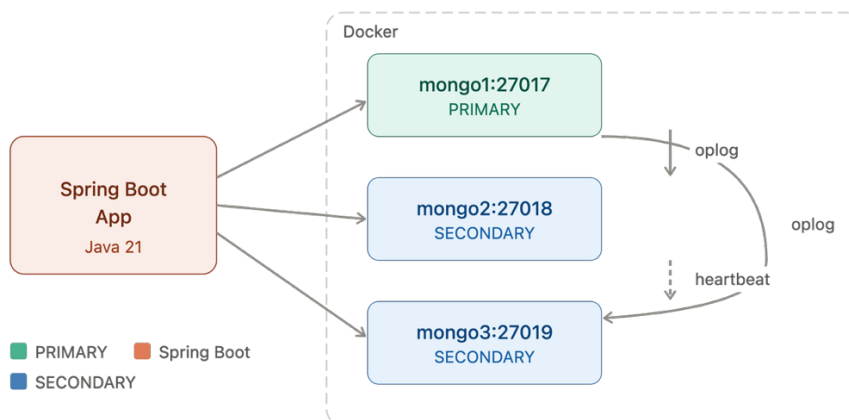
Sa druge strane, za sisteme koji zahtevaju ekstremne performanse pisanja ili horizontalnu skalabilnost i na nivou pisanja, Replica Set arhitektura može biti nedovoljna i trebalo bi razmotriti sharding arhitekturu, koja se može kombinovati sa Replica Set mehanizmom kako bi se postigla i horizontalna skalabilnost i visoka dostupnost.

6. Implementacija praktičnog dela

6.1. Arhitektura sistema

Praktični deo rada realizovan je kao Spring Boot aplikacija koja se povezuje na MongoDB Replica Set okruženje postavljeno u Docker kontejnerima. Sistem se sastoji od tri MongoDB čvora organizovana u Replica Set grupu i jedne Spring Boot aplikacije koja kroz REST API izlaže eksperimente za demonstraciju replikacije i visoke dostupnosti.

Svaki MongoDB čvor pokrenut je kao zasebni Docker kontejner. Čvor mongo1 sluša na portu 27017, čvor mongo2 na portu 27018, a čvor mongo3 na portu 27019. Svi čvorovi komuniciraju unutar Docker mreže koristeći interna DNS imena kontejnera, dok su portovi eksponirani na lokalnoj mašini domaćinu kako bi Spring Boot aplikacija mogla da pristupi svakom čvoru direktno. Ova konfiguracija portova ključna je za ispravno funkcionisanje MongoDB drajvera koji, nakon što otkrije topologiju Replica Set grupe, pokušava da komunicira sa svakim čvorom koristeći adresu i port koji su navedeni u konfiguraciji same Replica Set grupe.



Slika 8. Arhitektura implementiranog sistema - Spring Boot aplikacija i tri MongoDB čvora u Docker okruženju

Spring Boot aplikacija komunicira sa Replica Set grupom kroz tri različito konfigurisana MongoTemplate objekta. Primarni MongoTemplate konfigurisana je sa w:majority nivoom trajnosti i čitanjem sa primarnog čvora, sekundarni MongoTemplate konfigurisana je sa čitanjem sa sekundarnih čvorova, a treći MongoTemplate konfigurisana je sa w:1 nivoom trajnosti. Ova arhitektura omogućava demonstraciju različitih scenarija u okviru iste aplikacije.

6.2. Tehnološki stack

U implementaciji praktičnog dela korišćene su sledeće tehnologije:

Java 21 korišćen je kao primarni programski jezik. Java 21 je verzija sa dugoročnom podrškom (LTS) koja donosi niz unapređenja u pogledu performansi i novih jezičkih konstrukata, uključujući pattern matching i record tipove koji su korišćeni u implementaciji DTO klase.

Kotlin je korišćen za deo domenskih klasa koje su preuzete iz postojećeg projekta. Kotlin je programski jezik koji se kompajlira na JVM platformu i potpuno je interoperabilan sa Java kodom, što je omogućilo koegzistenciju Java i Kotlin klase unutar istog Maven projekta.

Spring Boot 4.0.6 je okvir koji čini osnovu aplikacije. Spring Boot obezbeđuje automatsku konfiguraciju, upravljanje zavisnostima i ugrađeni web server, što značajno smanjuje količinu boilerplate koda potrebnog za pokretanje aplikacije.

Spring Data MongoDB je modul Spring ekosistema koji pruža apstrakciju nad MongoDB Java drajverom. Kroz **MongoTemplate** klasu i **MongoRepository** interfejse, Spring Data MongoDB omogućava rad sa MongoDB kolekcijama na deklarativan način.

MongoDB 8.x je verzija MongoDB sistema korišćena u eksperimentima. MongoDB 8.x donosi unapređenja u pogledu performansi i stabilnosti u odnosu na prethodne verzije.

Docker i **Docker Compose** korišćeni su za postavljanje i upravljanje MongoDB kontejnerima. Docker Compose omogućava deklarativno definisanje višekontejnerskih okruženja kroz YAML konfiguracionu datoteku, što olakšava reprodukovanje okruženja na različitim mašinama.

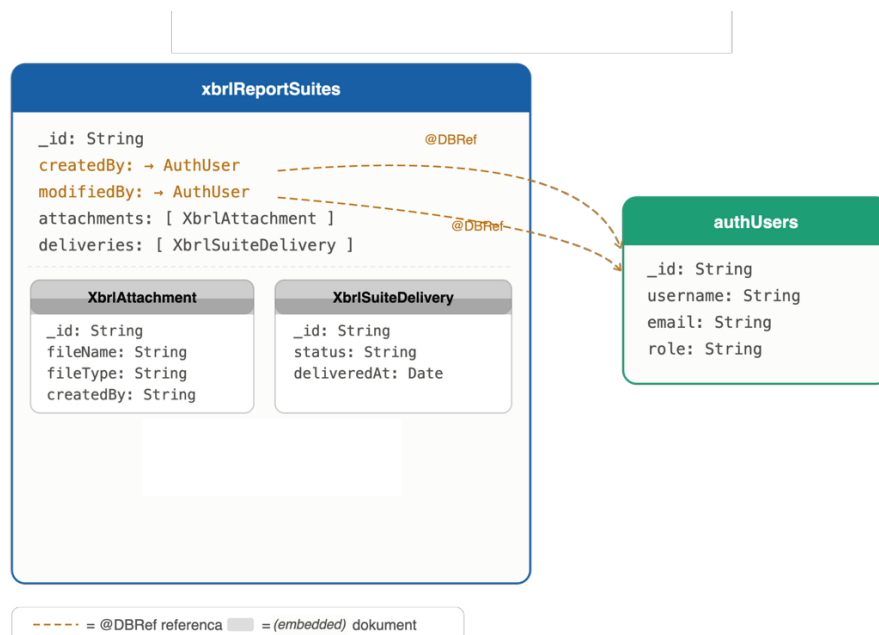
Maven je korišćen kao alat za upravljanje zavisnostima i izgradnju projekta. Maven POM datoteka definiše sve potrebne zavisnosti, uključujući Spring Boot starter module i MongoDB Java drajver.

Postman je korišćen za testiranje REST endpoint-a i prikupljanje rezultata eksperimenata.

6.3. Model sloj i domenski entiteti

Aplikacija koristi MongoDB dokumentni model za predstavljanje domenskih entiteta. Centralni entitet je **XbriReportSuite**, koji predstavlja XBRL izveštajni paket i čuva se kao zasebni MongoDB dokument u kolekciji **xbriReportSuites**. Entiteti **XbriAttachment**, **XbriSuiteDelivery** i **XbriSuiteDeliveryStatus** modelovani su kao ugniježđeni dokumenti unutar XbriReportSuite dokumenta, što odgovara pristupu agregata u dokumentnom modelu. Ovakvo modelovanje eliminiše potrebu za join operacijama pri čitanju podataka koji prirodno pripadaju zajedno.

Entitet **AuthUser** modelovan je kao zasebni dokument u kolekciji **authUsers** i referenciran je iz **XbriReportSuite** dokumenta kroz **@DBRef** anotaciju. Ova odluka opravdana je činjenicom da korisnici postoje nezavisno od izveštajnih paketa i mogu biti referencirani iz više različitih dokumenata.



Slika 9. Model domenskih entiteta sa prikazom ugniježđenih dokumenata i referenci

Sve MongoDB dokument klase nasleđuju apstraktnu klasu **BaseDocument** koja definiše zajedničko polje **id** tipa **String**, inicijalizovano kao UUID vrednost. Ova klasa zamenjuje prethodni JPA **BaseEntity** koji je koristio Hibernate-specifične anotacije.

Za potrebe eksperimenata replikacije, svi eksperimenti koriste posebnu kolekciju **replication_experiment** koja je odvojena od domenskih podataka aplikacije. Na taj način eksperimenti ne utiču na integritet poslovnih podataka. Eksperimenti su realizovani nad odvojenom kolekcijom `replication_experiment` kako bi se izbegao uticaj na integritet domenskih podataka. Ograničenje ovog pristupa jeste da izmerene latencije ne odražavaju veličinu realnih dokumenata. U produkcijskom okruženju sa stvarnim mrežnim kašnjenjem i većim dokumentima očekuju se proporcionalno veće vrednosti.

6.4. Point-in-Time Recovery

Replica Set okruženje konfigurisano je kroz Docker Compose datoteku koja definiše tri MongoDB servisa i jedan inicijalizacioni servis. Svaki MongoDB servis pokreće se sa parametrom **--replSet rs0** koji definiše naziv Replica Set grupe, i parametrom **--bind_ip_all** koji omogućava prihvatanje konekcija sa svih mrežnih interfejsa unutar Docker mreže.

Ključna konfiguracija koja omogućava ispravno funkcionisanje sistema na lokalnoj mašini jeste dodeljivanje jedinstvenih portova svakom čvoru. Čvor **mongo1** eksponirano je na portu 27017, **mongo2** na portu 27018, a **mongo3** na portu 27019. Svi hostname-ovi preslikani su na lokalnu adresu **127.0.0.1** u sistemskoj `/etc/hosts` datoteci, a razlika u portovima omogućava drajveru da razlikuje čvorove čak i kada komunicira sa lokalnom mašinom.

Nakon pokretanja kontejnera, Replica Set se inicijalizuje izvršavanjem **rs.initiate()** komande koja definiše konfiguraciju grupe sa tri člana i njihovim prioritetima. Čvoru **mongo1** dodeljen je prioritet 2, a čvorovima **mongo2** i **mongo3** prioritet 1, čime se osigurava da **mongo1** postane inicijalni primarni čvor.

```
→ mongodb-replica-set-ha git:(main) ✖ docker exec mongo1 mongosh --quiet --eval "rs.status().members.map(m => m.name + ' -> ' + m.stateStr)"
[
  'mongo1:27017 -> PRIMARY',
  'mongo2:27018 -> SECONDARY',
  'mongo3:27019 -> SECONDARY'
]
```

Slika 10. Izlaz `rs.status()` komande nakon uspešne inicijalizacije Replica Set grupe

Spring Boot aplikacija konfigurisana je da se povezuje na sve tri adrese čvorova kroz MongoDB URI koji navodi sve poznate članove grupe:

```
mongodb://mongo1:27017,mongo2:27018,mongo3:27019/?replicaSet=rs0
```

Navođenje svih članova u URI-u nije striktno obavezno jer MongoDB drajver automatski otkriva topologiju grupe, ali je preporučena praksa jer ubrzava inicijalno otkrivanje topologije i pruža rezervne adrese za slučaj da jedan od navedenih čvorova nije dostupan u trenutku uspostavljanja konekcije.

6.5. Demonstracija replikacije

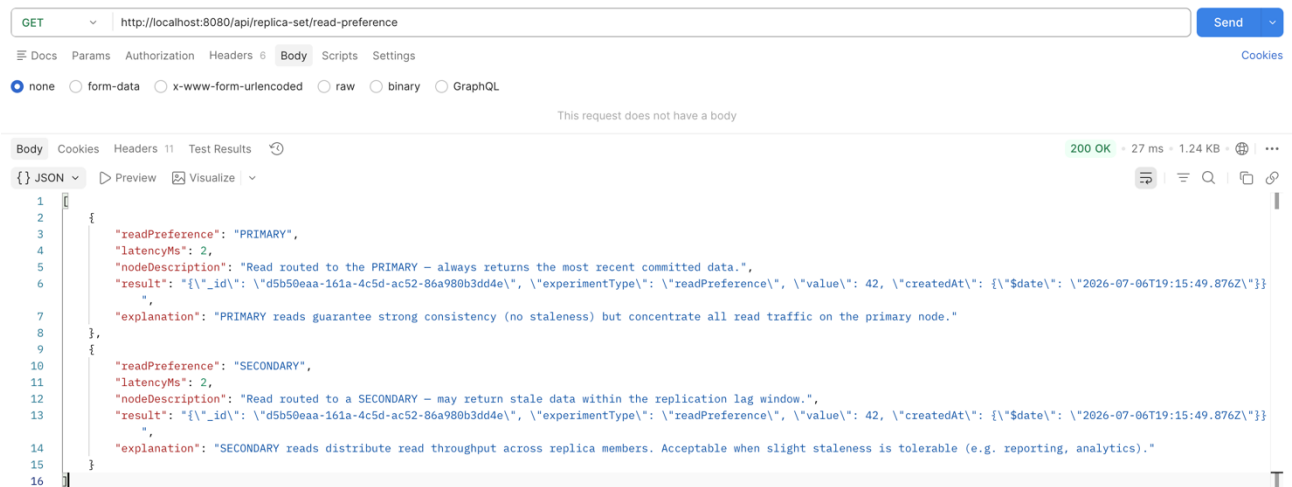
Eksperiment merenja kašnjenja replikacije implementiran je kao REST endpoint **/api/replica-set/replication-lag**. Metoda najpre umeće jedan testni dokument na primarni čvor koristeći **w:1** nivo trajnosti, čime se osigurava da operacija pisanja bude potvrđena pre nego što replikacija počne. Odmah nakon pisanja, metoda počinje da periodično proverava da li je dokument vidljiv na sekundarnom čvoru, sa intervalom od 5 ms između proverava.



Slika 11. Response endpoint-a /api/replica-set/replication-lag sa izmerenim kašnjenjem replikacije

Rezultati eksperimenta pokazali su kašnjenje replikacije od 15 ms, pri čemu je dokument bio vidljiv na sekundarnom čvoru već nakon prvog pokušaja čitanja. Ovo ukazuje na to da je replikacija bila završena između 10 ms i 15 ms nakon operacije pisanja. Ovakav rezultat očekivan je u lokalnom okruženju gde svi čvorovi komuniciraju unutar iste fizičke mašine bez mrežnog kašnjenja i bez opterećenja koje bi usporilo replikaciju.

Eksperiment sa Read Preference demonstrira razliku između čitanja sa primarnog i sekundarnog čvora kroz endpoint /api/replica-set/read-preference. Metoda umeće testni dokument sa **w:majority** nivoom trajnosti kako bi se osiguralo da je dokument repliciran pre nego što počne merenje čitanja. Nakon toga izvršava identičan upit čitanja dva puta, jednom sa primarnog i jednom sa sekundarnog čvora, i vraća rezultate sa izmerenim latencijama.

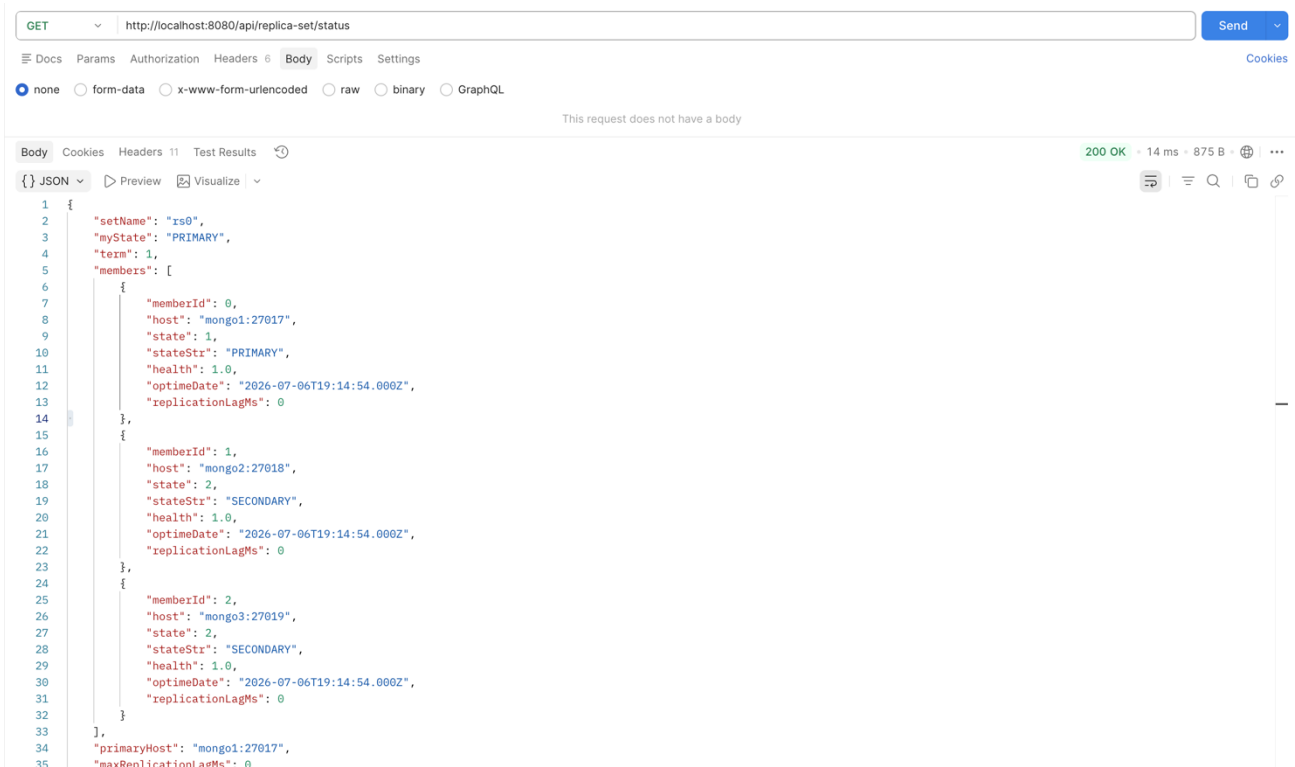


Slika 13. Response endpoint-a /api/replica-set/read-preference sa poređenjem latencija

Rezultati su pokazali identičnu latenciju od 2 ms za oba čvora, što je posledica lokalnog okruženja. Oba čvora vratila su identičan dokument, što potvrđuje da je replikacija bila završena u trenutku čitanja.

6.6. Demonstracija failover scenarija

Failover eksperiment implementiran je kao REST endpoint `/api/replica-set/failover` i predstavlja najsloženiji eksperiment u okviru praktičnog dela. Metoda prolazi kroz nekoliko koraka: identifikuje trenutnog primarnog čvora iz `rs.status()` odgovora, zaustavlja odgovarajući Docker kontejner putem **ProcessBuilder** mehanizma, a zatim periodično proverava `rs.status()` sve dok novi primarni čvor ne bude izabran.



Slika 14. Stanje Replica Set sistema pre failovera - mongo1 kao PRIMARY, term: 1

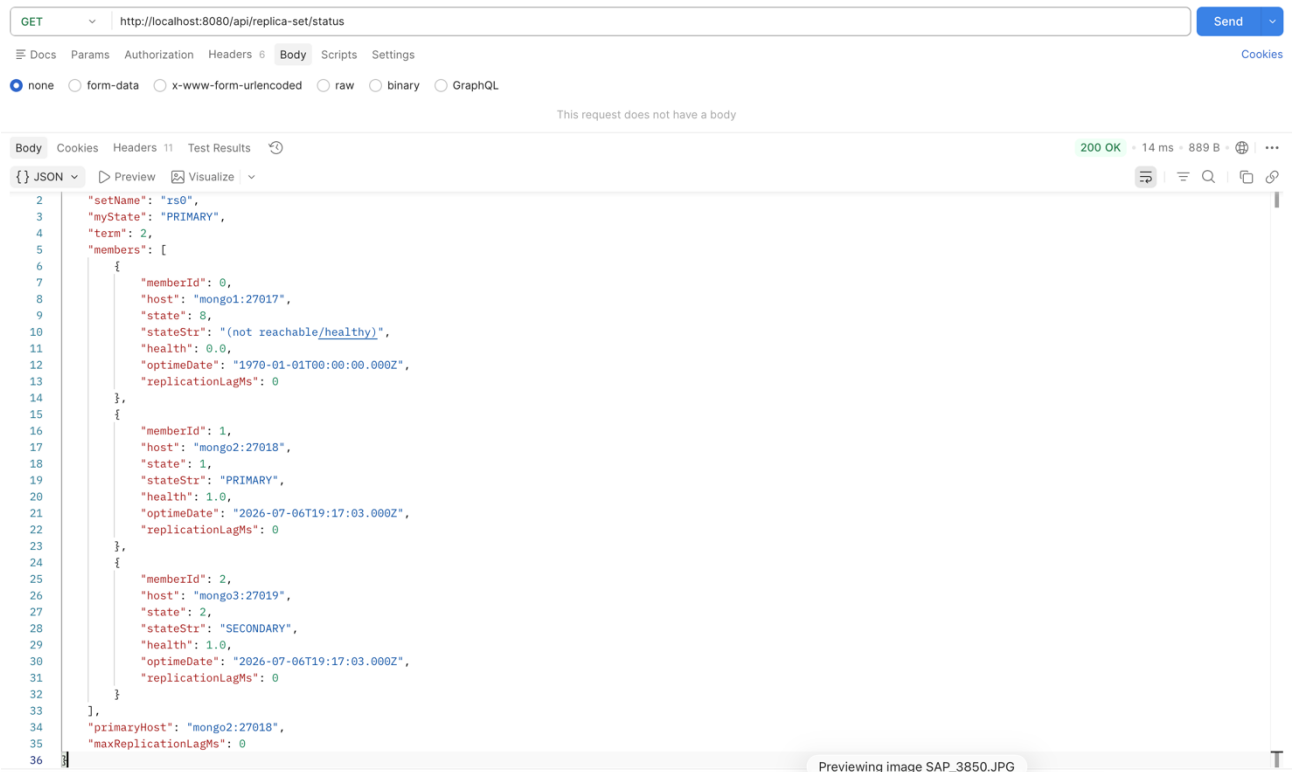
Pre pokretanja failovera, sistem je bio u sledećem stanju: čvor **mongo1:27017** imao je ulogu primarnog, dok su **mongo2:27018** i **mongo3:27019** bili sekundarni čvorovi. Svi čvorovi imali su vrednost **health: 1.0** i identičan **optimeDate**, što ukazuje na potpunu sinhronizaciju bez kašnjenja replikacije.

Nakon pozivanja endpoint-a, kontejner **mongo1** je zaustavljen, a sistem je automatski pokrenuo proces izbora novog primarnog čvora. Za svega 506 ms, čvor **mongo2:27018** izabran je kao novi primarni.



Slika 15. Rezultat failover procesa

Posebno je značajno primetiti da je vrednost polja **term** povećana sa 1 na 2. Ova vrednost predstavlja identifikator Raft termina i povećava se pri svakom novom procesu izbora, što omogućava čvorovima da razlikuju poruke iz različitih termina i spreči konflikte u situacijama kada stari primarni čvor pokušava da nastavi rad nakon što je već izabran novi.



Slika 16. Response endpoint-a /api/replica-set/status nakon failovera - mongo1 prikazan kao nedostupan

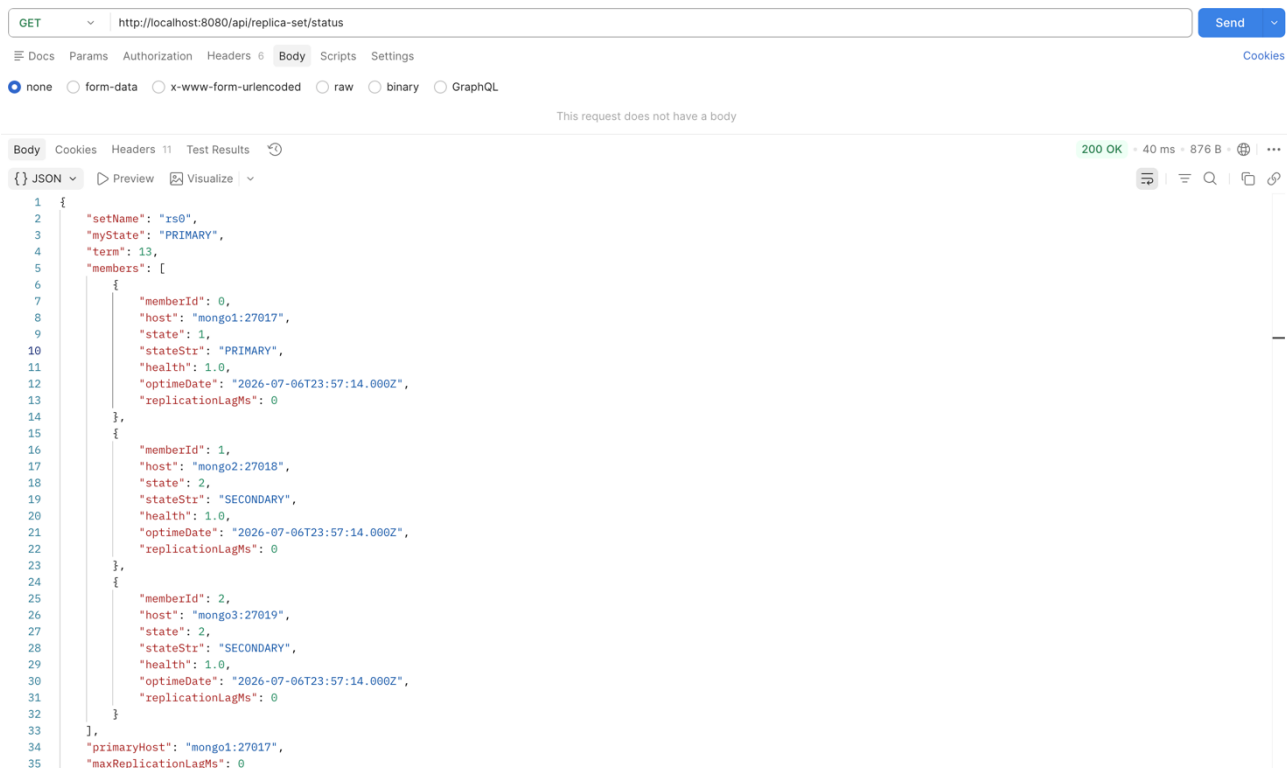
6.7. Demonstracija failback scenarija

Failback eksperiment implementiran je kao REST endpoint **/api/replica-set/failback**. Metoda identifikuje čvor sa vrednošću **health: 0.0** u **rs.status()** odgovoru, pokreće odgovarajući Docker kontejner i periodično proverava status dok čvor ne dostigne stanje **SECONDARY**.

Nakon pokretanja **mongo1** kontejnera, čvor je prošao kroz predviđeni niz stanja i za 2042 ms vratio se u stanje **SECONDARY**. Ukupno su bila potrebna četiri ciklusa provere sa intervalom od 500 ms.



Slika 18. Prikaz failback procesa



Slika 19. Response endpoint-a /api/replica-set/status nakon failbacka - sva tri čvora operativna, term: 13

Nakon završetka failback procesa, **rs.status()** pokazao je da je **mongo1** preuzeo ulogu primarnog čvora zahvaljujući višem prioritetu, a vrednost **term** povećana je na 3. Ovaj rezultat demonstrira da MongoDB Replica Set sistem po završetku oporavka vraća ulogu primarnog čvoru sa najvišim prioritetom, što je konzistentno sa konfiguracijom opisanom u poglavlju 6.4.

6.8. Merenja i rezultati eksperimenata

Tabela data u nastavku prikazuje zbirne rezultate svih sprovedenih eksperimenata.

Ekperiment	Merena veličina	Rezultat
Replikacija — kašnjenje	Vreme propagacije oplog unosa	15 ms
Replikacija — pokušaji čitanja	Broj provera sekundarnog čvora	1
Read Preference — PRIMARY	Latencija čitanja	2 ms
Read Preference — SECONDARY	Latencija čitanja	2 ms
Write Concern — w:1	Prosečna latencija pisanja (10 uzoraka)	3 ms
Write Concern — w:majority	Prosečna latencija pisanja (10 uzoraka)	1 ms
Failover — vreme izbora	Vreme od gašenja do novog PRIMARY	506 ms
Failover — pokušaji	Broj provera tokom election procesa	1
Failback — vreme oporavka	Vreme od pokretanja do SECONDARY stanja	2042 ms
Failback — pokušaji	Broj provera tokom oporavka	4

Slika 20. Zbirni rezultati eksperimenata

6.9. Analiza rezultata

Na osnovu dobijenih rezultata može se izvesti nekoliko zaključaka o ponašanju MongoDB Replica Set sistema u testiranom okruženju.

Kašnjenje replikacije od 15 ms u lokalnom Docker okruženju potvrđuje da je asinhrona replikacija u MongoDB sistemu izuzetno brza u uslovima bez mrežnog kašnjenja. Međutim, ovaj rezultat ne treba tumačiti kao reprezentativan za produkcijska okruženja. U okruženjima gde čvorovi komuniciraju kroz mrežu, kašnjenje replikacije direktno zavisi od mrežne latencije, opterećenja sistema i veličine operacija koje se repliciraju. U produkcijskim okruženjima tipično kašnjenje replikacije između čvorova u istom data centru kreće se od nekoliko milisekundi do nekoliko desetina milisekundi, dok između geografski udaljenih čvorova može biti i nekoliko stotina milisekundi..

Rezultati Write Concern eksperimenta pokazuju naizgled neočekivani odnos: **w:majority** je pokazao nižu latenciju od **w:1**. Ovaj rezultat direktna je posledica lokalnog okruženja u kome komunikacija između čvorova ne uključuje stvarno mrežno kašnjenje. U produkcijskom okruženju sa fizički odvojenim serverima, **w:majority** uvek rezultuje višom latencijom jer zahteva dodatni mrežni prenos ka sekundarnim čvorovima i natrag. Merenja sprovedena u ovom radu stoga ilustruju koncept razlike između ova dva nivoa trajnosti, ali ne odražavaju apsolutne vrednosti koje bi se mogle očekivati u produkciji.

Vreme failovera od 506 ms je izuzetno kratko i posledica je kombinacije nekoliko faktora. Čvorovi koji komuniciraju unutar iste mašine detektuju nedostupnost primarnog gotovo trenutno, bez kašnjenja koje bi unelo sporija mrežna veza. Pored toga, u konfiguraciji sa tri čvora samo dva čvora moraju razmenjivati poruke za glasanje, što minimizuje vreme potrebno za postizanje konsenzusa. U produkcijskim okruženjima sa mrežnim kašnjenjem, vreme failovera tipično se kreće između 10 i 30 sekundi, uzimajući u obzir podrazumevanu vrednost **electionTimeoutMillis** parametra od 10 sekundi.

Vreme oporavka od 2042 ms tokom failback eksperimenta odrazilo je minimalnu količinu posla koji je čvor morao da obavi po ponovnom pokretanju, budući da je bio nedostupan svega nekoliko sekundi i da je morao da preuzme samo manji broj oplog unosa. U slučaju dužeg perioda nedostupnosti, proces oporavka bi trajao proporcionalno duže, a u ekstremnim slučajevima kada čvor zaostane toliko da propuštene operacije više nisu u oplog-u, MongoDB bi pokrenuo inicijalni sync koji može trajati satima za velike skupove podataka.

Može se zaključiti da eksperimenti uspešno demonstriraju ključne aspekte MongoDB Replica Set arhitekture: asinhronu replikaciju sa merljivim kašnjenjem, razliku između različitih Read Preference i Write Concern konfiguracija, kao i automatski failover i oporavak sistema. Ograničenja uočenih eksperimenata su pre svega posledica lokalnog Docker okruženja koje ne modeluje realne mrežne uslove, ali konceptualna demonstracija ostaje validna i pruža osnovu za razumevanje ponašanja sistema u produkcijskim uslovima. Dodatno ograničenje predstavlja korišćenje odvojene kolekcije replication_experiment sa minimalnim testnim dokumentima umesto realnih domenskih podataka. Eksperimenti nad većim dokumentima u produkcijskom okruženju sa stvarnim mrežnim kašnjenjem rezultovali bi merljivo većim vrednostima latencije i kašnjenja replikacije.

7. Zaključak

U ovom radu opisana je arhitektura MongoDB Replica Set sistema i mehanizmi koji stoje u osnovi visoke dostupnosti distribuiranih baza podataka. Na teorijskom nivou analizirani su ključni koncepti: asinhrona replikacija kroz oplog mehanizam, heartbeat protokol za praćenje zdravlja čvorova, Raft-zasnovani algoritam za izbor primarnog čvora, kao i konfiguracioni parametri Read Preference, Write Concern i Read Concern koji definišu kompromise između performansi, konzistentnosti i trajnosti podataka. Prikazane su i prednosti i ograničenja Replica Set arhitekture, čime je data uravnotežena slika koja omogućava procenu prikladnosti ovog pristupa za konkretne scenarije primene.

Na praktičnom nivou implementirana je Spring Boot aplikacija koja se povezuje na tročlani MongoDB Replica Set podignut u Docker okruženju. Kroz šest REST endpoint-a demonstrirani su i izmereni ključni scenariji: kašnjenje replikacije između primarnog i sekundarnog čvora, razlika u ponašanju sistema pri različitim Read Preference konfiguracijama, poređenje latencije pisanja za **w:1** i **w:majority** nivoe trajnosti, automatski failover sa merenjem vremena izbora novog primarnog čvora i automatski oporavak sistema kroz failback proces.

Rezultati eksperimenata potvrdili su teorijska razmatranja. Kašnjenje replikacije izmereno je na 15 ms u lokalnom okruženju, što potvrđuje asinhroni karakter replikacije i postojanje kratkog prozora tokom kojeg sekundarni čvorovi mogu vraćati zastarele podatke. Failover proces završen je za svega 506 ms, što demonstrira sposobnost MongoDB Replica Set sistema da u optimalnim uslovima obnovi dostupnost pisanja u vremenu kraćem od jedne sekunde. Failback proces, kojim se oporavljeni čvor vratio u ulogu sekundarnog, trajao je 2042 ms i odvijao se u potpunosti automatski bez potrebe za ručnom intervencijom.

Može se zaključiti da je cilj rada ostvaren. MongoDB Replica Set arhitektura demonstrirala je sve ključne osobine sistema visoke dostupnosti: redundantnost podataka, automatski failover, automatski oporavak i fleksibilnu konfiguraciju nivoa konzistentnosti. Eksperimenti su pokazali da ovi mehanizmi funkcionišu pouzdano i da su parametri kao što su vreme failovera i kašnjenje replikacije merljivi i predvidivi.

Ograničenja sprovedenih eksperimenata vezana su pre svega za lokalno Docker okruženje koje ne odražava realne mrežne uslove. Rezultati kao što su latencije pisanja i čitanja od svega 1-3 ms, kao i vreme failovera od 506 ms, posledica su odsustva mrežnog kašnjenja između čvorova koji se nalaze na istoj fizičkoj mašini. U produkcijskim okruženjima sa geografski raspoređenim čvorovima ove vrednosti bile bi značajno veće i zavisile bi od konkretnih mrežnih uslova.

Kao moguća unapređenja sistema mogu se navesti nekoliko pravaca. Uvođenje sharding arhitekture uz Replica Set omogućilo bi horizontalnu skalabilnost i na nivou pisanja, što bi proširilo primenu sistema na scenarije sa ekstremnim opterećenjem. Konfiguracija čvorova u različitim geografskim lokacijama omogućila bi merenje realnih mrežnih latencija i demonstraciju zone sharding mehanizma. Dodavanje monitoring komponente, kao što je MongoDB Atlas ili Prometheus sa odgovarajućim exporterom, pružilo bi kontinuirani uvid u zdravlje sistema i istorijske podatke o kašnjenju replikacije. Konačno, eksperimenti bi mogli biti prošireni testovima opterećenja koji bi pokazali kako Replica Set sistem reaguje na visok broj istovremenih zahteva u kombinaciji sa failover scenarijem.

Budući pravci razvoja u oblasti replikacije i visoke dostupnosti u MongoDB sistemu uključuju unapređenja u pogledu brzine oporavka kroz mehanizam accelerated elections, poboljšanja Change Streams funkcionalnosti koja omogućava reaktivno praćenje promena u podacima u realnom vremenu, kao i integraciju sa oblak platformama kroz MongoDB Atlas koji automatizuje upravljanje Replica Set okruženjem i pruža globalnu distribuciju podataka kroz Global Clusters funkcionalnost.

LITERATURA

- [1] MongoDB, Inc. MongoDB Manual - Replication
<https://www.mongodb.com/docs/manual/replication/>
- [2] MongoDB, Inc. MongoDB Manual - Replica Set Members
<https://www.mongodb.com/docs/manual/core/replica-set-members/>
- [3] MongoDB, Inc. MongoDB Manual - Replica Set Elections
<https://www.mongodb.com/docs/manual/core/replica-set-elections/>
- [4] MongoDB, Inc. MongoDB Manual - Replica Set Oplog
<https://www.mongodb.com/docs/manual/core/replica-set-oplog/>
- [5] MongoDB, Inc. MongoDB Manual - Read Preference
<https://www.mongodb.com/docs/manual/core/read-preference/>
- [6] MongoDB, Inc. MongoDB Manual - Write Concern
<https://www.mongodb.com/docs/manual/reference/write-concern/>
- [7] MongoDB, Inc. MongoDB Manual - Read Concern
<https://www.mongodb.com/docs/manual/reference/read-concern/>
- [8] MongoDB, Inc. MongoDB Manual - Replica Set High Availability
<https://www.mongodb.com/docs/manual/core/replica-set-high-availability/>
- [9] Spring. Spring Data MongoDB - Reference Documentation
<https://docs.spring.io/spring-data/mongodb/docs/current/reference/html/>
- [10] MongoDB University. M103: Basic Cluster Administration
<https://learn.mongodb.com/>